

Введение

Шахматные движки - это программы, которые анализируют позицию на доске и находят наилучший ход. Современные движки играют значительно сильнее любого человека: рейтинг лучших программ превышает 3500 эло, тогда как рейтинг сильнейших гроссмейстеров мира находится в районе 2800. Но как он перебирает миллионы ходов за секунду и выбирает лучший?

В этой статье подробно разберём шахматный движок под капотом. В качестве основного примера мы будем использовать **Stockfish** - самый популярный и наиболее изученный open-source движок в мире.

1. Что такое Stockfish?

Stockfish - это бесплатный, открытый шахматный движок, написанный на C++. Он был создан в ноябре 2008 года как форк движка Glaurung. С тех пор над ним работают десятки разработчиков по всему миру, а сама программа регулярно обновляется и улучшается.

Stockfish написан на C++ и компилируется под Windows, Linux, macOS, iOS и Android. Он поддерживает протокол **UCI (Universal Chess Interface)** - стандартный способ общения между движком и графическим интерфейсом. Движок не имеет собственного визуального интерфейса: он общается с GUI через стандартный ввод/вывод (stdin/stdout), получая команды и отправляя ответы в текстовом виде.

2. Представление шахматной доски - Bitboard

Прежде чем анализировать позицию, движок должен как-то хранить её в памяти. Наивный подход - хранить двумерный массив 8×8, где каждая клетка содержит тип фигуры. Но Stockfish использует значительно более эффективный метод - **битборды**.

Что такое битборд?

Шахматная доска содержит ровно 64 клетки. Это ровно столько же, сколько битов в 64-битном целом числе (uint64_t в C++). Идея битборда: **каждый бит числа соответствует одной клетке доски**. Если бит равен 1 - фигура на этой клетке есть, если 0 - нет.

В Stockfish хранится не один битборд, а сразу несколько - по одному для каждого типа фигур каждого цвета. Например:

- whitePawns - 64-битное число, в котором 1 стоит на позициях белых пешек
- blackKnights - аналогично для чёрных коней
- whiteRooks - для белых ладей
- ... и т.д.

Для начальной позиции битборд белых пешек выглядит так:

```
0 0 0 0 0 0 0 0
0 0 0 0 0 0 0 0
0 0 0 0 0 0 0 0
0 0 0 0 0 0 0 0
0 0 0 0 0 0 0 0
0 0 0 0 0 0 0 0
1 1 1 1 1 1 1 1  <- вторая горизонталь, все пешки белых
0 0 0 0 0 0 0 0
```

В виде числа это 0x000000000000FF00 (гексадецимальная запись).

Зачем это нужно? Побитовые операции

Главное преимущество битбордов - возможность выполнять операции над всеми фигурами одновременно, используя **побитовые операции** процессора. Это чрезвычайно быстро.

Чтобы сдвинуть все белые пешки на одну клетку вперёд (т.е. получить множество возможных ходов-продвижений), достаточно выполнить битовый сдвиг:

```
whitePawns << 8  // все пешки сдвигаются на одну клетку вперёд
```

Чтобы найти все занятые клетки (любой фигурой), достаточно объединить все битборды операцией OR:

```
occupancy = whitePawns | whiteKnights | ... | blackKing;
```

Чтобы найти клетки, атакованные белыми пешками, используют сдвиги по диагоналям:

```
pawnAttacks = (whitePawns << 7) | (whitePawns << 9);
```

Все эти операции выполняются за один такт процессора - вне зависимости от количества фигур. Это в десятки раз быстрее, чем перебор элементов массива.

3. Генерация ходов (Move Generation)

Следующий шаг после хранения позиции - генерация всех возможных ходов для текущей позиции.

Типы фигур и их ходы

Фигуры делятся на две категории по способу генерации ходов:

Фигуры с фиксированными паттернами (пешки, кони, король): множество атакуемых ими клеток фиксировано и зависит только от их текущей позиции, но не от расположения других фигур. Для них Stockfish использует заранее вычисленные таблицы (lookup tables): массивы, индексированные по номеру клетки, которые возвращают битборд атакуемых клеток.

Скользящие фигуры (слоны, ладьи, ферзи): их ходы зависят от расположения других фигур (они не могут «прыгать» через них). Для них используется более сложная техника - Magic Bitboards.

Magic Bitboards - магические числа

Magic Bitboards - остроумный алгоритм для быстрого вычисления ходов скользящих фигур. Его идея: для каждой клетки доски и каждого типа фигуры (ладья или слон) существует заранее вычисленное "магическое число". Когда мы умножаем битборд занятых клеток на это число и берём старшие биты результата - получается индекс в таблице атак.

```
// Пример для ладьи на клетке sq
uint64_t magic = RookMagics[sq];
uint64_t mask = RookMasks[sq];
uint64_t key = ((occupancy & mask) * magic) >> 52;
uint64_t attacks = RookAttacks[sq][key];
```

По сути это **хеш-функция**: она очень быстро отображает конфигурацию фигур на доске в готовый набор возможных ходов. Магические числа подбираются один раз при инициализации методом полного перебора или хранятся готовыми в коде.

Представление хода

Каждый ход в Stockfish - это **16-битное целое число**. В нём закодированы:

- **from** (6 бит): клетка-источник, номер от 0 до 63
- **to** (6 бит): клетка-назначение, номер от 0 до 63
- **флаги** (4 бита): тип хода (рокировка, взятие на проходе, превращение пешки и т.д.)

Такое компактное представление позволяет хранить огромное количество ходов в кэше процессора, что существенно ускоряет работу.

4. Поиск лучшего хода - алгоритм Minimax

Итак, у нас есть позиция и список всех возможных ходов. Как теперь выбрать лучший? Ключевая идея - построить дерево вариантов и оценить все возможные позиции.

Игровое дерево

Шахматная партия - это дерево, где корень - текущая позиция, а ветви - возможные ходы. После первого хода белых возникают n дочерних позиций. После ответного хода чёрных - каждая из них разветвляется ещё на m позиций. И так далее.

Проблема в том, что количество позиций растёт экспоненциально. В шахматах в среднем около 35 возможных ходов в позиции - это число называют **коэффициентом ветвления (branching factor)** и обозначают b . Значит, на глубине 6 ходов нужно оценить около $35^6 \approx 1,8$ миллиарда позиций.

Полный перебор невозможен $\rightarrow$ нужны умные алгоритмы.

Алгоритм Minimax

Minimax - базовый алгоритм для игр с двумя игроками. Его логика:

- Белые хотят **максимизировать** оценку позиции
- Чёрные хотят **минимизировать** оценку
- На каждом уровне дерева ход делает то максимизирующая, то минимизирующая сторона
- Конечные позиции оцениваются функцией оценки (evaluation function)

```
def minimax(position, depth, isMaximizing):
    if depth == 0:
        return evaluate(position) # оцениваем позицию

    if isMaximizing: # ход белых, максимизируем
        best = -infinity
        for move in generateMoves(position):
            score = minimax(makeMove(position, move), depth-1, False)
            best = max(best, score)
        return best
    else: # ход чёрных, минимизируем
        best = +infinity
        for move in generateMoves(position):
            score = minimax(makeMove(position, move), depth-1, True)
            best = min(best, score)
        return best
```

Альфа-бета отсечение

Альфа-бета отсечение - самое важное улучшение Minimax. Оно позволяет «обрезать» ветви дерева, которые заведомо не влияют на итоговый результат.

В процессе поиска отслеживаются два значения:

- **Alpha (α)** - лучшая оценка, которую белые уже могут гарантировать (нижняя граница)
- **Beta (β)** - лучшая оценка, которую чёрные уже могут гарантировать (верхняя граница)

Если в какой-то момент текущая ветвь не может улучшить результат (оценка выходит за рамки $[\alpha, \beta]$), мы прекращаем её анализировать - это и есть «отсечение».

Пример: мы нашли ход, дающий оценку +1.0. Теперь рассматриваем другой ход, и после первого ответа противника уже видите оценку -0.5. Нет смысла смотреть остальные ответы: этот ход хуже первого, и мы его «обрезаем».

```
def alphaBeta(position, depth, alpha, beta, isMaximizing):
    if depth == 0:
        return evaluate(position)

    if isMaximizing:
        for move in generateMoves(position):
            score = alphaBeta(makeMove(position, move), depth-1,
                              alpha, beta, False)
            alpha = max(alpha, score)
            if alpha >= beta:
                break # бета-отсечение
        return alpha
    else:
        for move in generateMoves(position):
            score = alphaBeta(makeMove(position, move), depth-1,
                              alpha, beta, True)
            beta = min(beta, score)
            if beta <= alpha:
                break # альфа-отсечение
        return beta
```

Математический результат: при идеальном порядке ходов альфа-бета сокращает число узлов с b^n до $b^{\frac{n}{2}}$, где b - количество ходов, n - глубина. Это означает, что за то же время можно просчитать **вдвое большую глубину** - например, перейти с 10 до 15-18 полуходов.

Порядок ходов

Из алгоритма видно: чем раньше рассматривается лучший ход, тем больше ветвей получится отсечь. Поэтому порядок ходов критически важен. Stockfish использует несколько эвристик:

- **Killer moves** - ходы, которые вызвали отсечение в соседних ветвях на той же глубине. Хранятся в специальной таблице.
- **History heuristic** - история успеха каждого хода: ходы, которые чаще вызывали отсечения, рассматриваются первыми.
- **MOV-LVA** (Most Valuable Victim - Least Valuable Attacker) — при взятиях первыми рассматриваются взятия, где самая ценная фигура берётся наименее ценной.
- **Ход из таблицы транспозиций** - если позиция уже встречалась, рассматривается первым.

5. Дополнительные техники поиска

Помимо базовой альфа-беты, современные движки используют множество дополнительных техник.

Таблица транспозиций

Одна и та же позиция может быть достигнута разными последовательностями ходов. Нет смысла анализировать одно и то же дважды.

Таблица транспозиций - хеш-таблица, в которой хранятся уже вычисленные оценки позиций. Когда движок встречает позицию повторно, он берёт готовый результат.

Для идентификации позиции используется **хеш Zobриста**. При инициализации для каждой комбинации "фигура x клетка" генерируется случайное 64-битное число. Хеш позиции - XOR всех таких чисел для всех фигур на доске:

```
uint64_t hash = 0;
for (auto [piece, square] : piecesOnBoard) {
    hash ^= zobristTable[piece][square];
}
```

Ключевое свойство XOR: при ходе фигуры с клетки A на клетку B хеш пересчитывается **инкрементально** - не заново, а за два XOR:

```
hash ^= zobristTable[piece][from_square]; // убираем фигуру
hash ^= zobristTable[piece][to_square];   // ставим фигуру
```

В Stockfish таблица транспозиций глобальна и разделяется между всеми потоками. Для скорости используется **lock-free доступ** - потоки могут одновременно читать и писать, допуская редкие гонки данных в обмен на скорость.

Итеративное углубление

Вместо того чтобы сразу искать на глубину N , движок сначала ищет на глубину 1, потом на 2, потом на 3 и так далее - до тех пор, пока не кончится время.

Это кажется расточительным, но поиск на глубину N занимает примерно в b раз больше, чем на $N-1$. Поэтому суммарное время всех предыдущих итераций меньше, чем время последней. При этом предыдущие итерации дают ценную информацию - лучший ход с меньшей глубины хранится в таблице транспозиций и рассматривается первым на следующей итерации, улучшая порядок ходов.

Нулевой ход

Движок делает "нулевой ход" - передаёт ход противнику, не делая хода сам. Если даже с такой форой противник не может улучшить позицию ниже бета, значит текущий узел можно смело отсечь.

Более понятно: если мы "пропустим" наш ход и противник всё равно не сможет угрожать нам ничем серьёзным - значит наша позиция настолько хороша, что её можно не анализировать глубже. Null-move отключается в эндшпиле и в позициях под шахом.

Поиск до спокойной позиции

Обычный поиск останавливается на заданной глубине и вызывает функцию оценки. Но что если на этой глубине идёт размен фигур? Оценка будет неточной.

Quiescence Search: на терминальной глубине движок продолжает поиск, но рассматривает только взятия и шахи - до тех пор, пока позиция не станет "тихой". Это предотвращает **горизонтный эффект** - когда движок не видит тактики, выходящей за горизонт основного поиска.

Late Move Reductions

Ходы, которые рассматриваются поздно в списке, с высокой вероятностью плохие. **LMR** - техника, при которой такие ходы просматриваются на меньшую глубину. Если сокращённый поиск показывает интересный результат, движок переделывает поиск на полную глубину.

6. Функция оценки

Функция оценки - сердце движка: она численно измеряет "хорошесть" позиции. Положительное значение означает преимущество белых, отрицательное - преимущество чёрных. Единица измерения - **centipawn (cp)**: оценка +100cp означает примерно пешечный перевес белых.

Классическая оценка (до Stockfish 16)

До 2023 года Stockfish использовал классическую оценку - набор вручную написанных правил:

Фактор	Описание	Пример
Материал	Количество и тип фигур	Ферзь = 900ср, ладья = 500ср, слон/конь = 330ср, пешка = 100ср
Таблицы позиций (PST)	Бонусы/штрафы за положение фигуры на доске	Конь в центре лучше, чем на краю
Структура пешек	Качество пешечной структуры	Штраф за сдвоенные и изолированные пешки; бонус за проходную
Безопасность короля	Насколько защищён король	Штраф если рядом с королём мало защитников
Подвижность фигур	Количество доступных ходов	Слон на длинной диагонали получает бонус
Взаимодействие фигур	Пары слонов, батареи ладей	Два слона сильнее двух коней в открытой позиции

Все параметры настраивались с помощью **fishtest** - системы тестирования, где тысячи компьютеров по всему миру играют миллионы партий для проверки каждого изменения (метод SPSA).

Нейросетевая оценка

В августе 2020 года в Stockfish была интегрирована принципиально новая функция оценки - **NNUE** (Efficiently Updatable Neural Network). С выходом Stockfish 16 в июле 2023 года классическая оценка была полностью удалена.

NNUE была изначально разработана для японских шахмат (сёги) Ю Насу в 2018 году. Ключевая идея: использовать нейронную сеть для оценки, но организовать её так, чтобы при каждом ходе пересчитывалась только изменившаяся часть сети.

Как работает NNUE

1. Входные признаки. Позиция описывается разреженным вектором признаков, основанных на расположении короля и других фигур (архитектура HalfKAv2_hm). Для каждой стороны используется 41 024 входных признака, но почти все из них равны нулю - ненулевыми являются только те, которые соответствуют реально стоящим фигурам.

2. Архитектура сети. Stockfish использует две сети одновременно:

- **Малая сеть:** скрытый слой 128 нейронов - используется когда материальный дисбаланс велик (позиция и так очевидна)
- **Большая сеть:** скрытый слой 1024 нейрона - используется для сбалансированных позиций, требующих тонкой оценки

3. Инкрементальные обновления. При ходе фигуры изменяется лишь небольшое количество входных признаков. NNUE пересчитывает только изменившиеся части первого (самого большого) слоя - это называется **аккумулятором**. Остальные слои пересчитываются целиком, но они намного меньше. Именно это и делает сеть эффективно обновляемой.

4. Квантизация. Для скорости все веса сети хранятся в виде 8- и 16-битных целых чисел (вместо обычных 32-битных float). Это позволяет использовать **SIMD-инструкции** процессора (AVX2, AVX-512), которые обрабатывают сразу 32 или 64 значения за один такт.

Результат: NNUE оценивает позиции значительно точнее классической функции. По сравнению со Stockfish 11, Stockfish 12 с NNUE выигрывал в 10 раз больше партий, чем проигрывал.

7. Многопоточность и протокол UCI

Многопоточный поиск

Современные компьютеры имеют несколько ядер процессора. Stockfish использует их все одновременно с помощью метода **Lazy SMP** (Symmetric Multiprocessing).

Несколько потоков запускают один и тот же алгоритм поиска параллельно, начиная с корня. Они делятся одной таблицей транспозиций. Потоки не синхронизируются явно, они просто работают независимо и получают пользу от результатов, которые другие потоки успели записать в ТТ. Разные потоки исследуют дерево в разном порядке, и в целом это ускоряет поиск. Stockfish поддерживает до 512 потоков одновременно.

Протокол UCI

UCI (Universal Chess Interface) - текстовый протокол общения между движком и GUI. Движок читает команды из stdin и пишет ответы в stdout.

Базы окончаний

Для позиций с малым количеством фигур (5–7 фигур) существуют **базы окончаний** - предвычисленные результаты для ВСЕХ возможных позиций с данным набором фигур. Stockfish поддерживает Syzygy tablebases.

Когда количество фигур падает до значения, покрытого базой, движок просто смотрит в таблицу вместо поиска. Это гарантирует идеальную игру в эндшпиле. Syzygy для 7 фигур занимает около 18 ТБ дискового пространства.

8. Архитектура движка целиком

Компонент	Назначение	Ключевая идея
Bitboards	Хранение позиции	64-битное число = 64 клетки; побитовые операции
Magic Bitboards	Ходы скользящих фигур	Хеш-функция для мгновенного вычисления атак
Alpha-Beta	Поиск лучшего хода	Отсечение ненужных ветвей дерева
Move Ordering	Ускорение Alpha-Beta	Лучшие ходы - первыми
Transposition Table	Кэш позиций	Не считать одно и то же дважды
Zobrist Hashing	Идентификация позиции	XOR случайных чисел, инкрементальное обновление
NNUE	Оценка позиции	Нейросеть с инкрементальными обновлениями
Quiescence Search	Точность оценки	Дотянуть до тихой позиции
Null Move Pruning	Ускорение поиска	Пропуск хода как форвардное отсечение
Iterative Deepening	Управление временем	Поиск нарастающей глубины
Lazy SMP	Многопоточность	Несколько потоков + общая таблица транспозиций

Заключение

Bitboard-представление доски, альфа-бета отсечение как ядро поиска, таблица транспозиций для кэширования. Все они не менялись десятилетиями.

Революционным прорывом стало появление NNUE в 2020 году: нейросетевая функция оценки, которая работает достаточно быстро для интеграции в классический поиск, дала скачок в силе игры, сопоставимый с несколькими годами предыдущего прогресса.

Источники и материалы для дальнейшего изучения

- **Исходный код Stockfish:** github.com/official-stockfish/Stockfish
- **Официальная документация:** official-stockfish.github.io/docs
- **Chess Programming Wiki:** [chessprogramming.org](https://www.chessprogramming.org/)
- **DeepWiki Stockfish:** deepwiki.com/official-stockfish/Stockfish
- **Статья о NNUE:** stockfishchess.org/blog/2020/introducing-nnue-evaluation